

Clase 11 · Modelado OO + Diagrama de Contexto

Modelación y Diseño de Sistemas 2026 · UNC-ESCMB-AUSI

Contenidos de hoy

- Modelado Orientado a Objetos
- Diagrama de Contexto + Actores
- Primer paso del bloque UML Análisis

Caso Hilo Conductor

ChangoMás · Cajas de Autoservicio

Ing. Fernando Bono · linkedin.com/in/fernandobono/



Agenda de la clase

90 minutos · 3 bloques + ejercicio

1

BLOQUE 1 · Modelado OO ~20 min

- ¿Para qué modelar?
- Conceptos OO (abstracción, encapsulamiento, herencia, polimorfismo)
- Clases, objetos, responsabilidades
- Intro a UML y sus diagramas

2

BLOQUE 2 · Contexto + Actores ~30 min

- ¿Qué es el Diagrama de Contexto?
- Elementos: sistema, actores, externos
- Actor primario vs secundario
- Cómo identificarlos + antipatrones

3

BLOQUE 3 · ChangoMás + Taller ~40 min

- Ejemplo ChangoMás (sistema + actores)
- Diagrama dibujado en vivo
- Ejercicio en clase sobre OAS propio
- Cierre y próximos pasos

Recap — Bloque Scrum (Clases 8-10)

¿Con qué venimos? Puente al nuevo bloque

YA TENEMOS

- OAS 2024 convertido a Product Backlog
- Épicas: Self-Checkout · Control y Seguridad · Gestión Operativa
- Historias de usuario INVEST con criterios de aceptación
- Formato: Como <rol>, quiero <algo> para <beneficio>
- Estimación (story points / planning poker)
- Sprint Planning + DoR / DoD + Azure DevOps
- Ceremonias Scrum aplicadas sobre el caso

AHORA EMPEZAMOS

→ BLOQUE UML ANÁLISIS

- Modelamos **ANTES** de diseñar y codear
- Las HU cuentan **QUÉ** quiere cada rol
- El modelo cuenta **CÓMO** se relacionan los conceptos y actores del dominio
- Primer paso: ubicar el sistema en su entorno → **Diagrama de Contexto**



¿Para qué modelar?

> "Construimos modelos para comprender mejor el sistema que estamos desarrollando. Un único modelo nunca alcanza."



Conocer el dominio

Captar el vocabulario y las reglas del negocio antes de escribir código.



Descubrir requisitos

Los huecos en el modelo revelan preguntas que nadie se había hecho.



Comunicar

Un diagrama es lenguaje común entre devs, analistas, usuarios y gerentes.



Reducir complejidad

Elegimos un punto de vista y dejamos afuera lo irrelevante para ese foco.



Documentar decisiones

El modelo fija lo acordado y evita revivir discusiones viejas.



Explorar alternativas

Probar diseños en papel cuesta mucho menos que probarlos en código.

Modelado Orientado a Objetos

Los 4 conceptos base del paradigma

Abstracción

Identificar las características esenciales de una cosa del mundo, ignorando detalles irrelevantes para el problema.

Ej. Cliente → DNI, nombre, medio de pago. No: color de remera.

Encapsulamiento

Empaquetar datos + operaciones en una unidad y exponer solo lo necesario. Esconde el *cómo*.

Ej. La Transacción sabe cómo calcular su total; nadie de afuera toca sus items.

Herencia

Una clase hija reutiliza y extiende lo definido en una clase padre. Relación "es-un".

Ej. Asistente y Auxiliar son tipos de Empleado. Ambos tienen legajo.

Polimorfismo

La misma operación puede comportarse distinto según el tipo concreto del objeto.

Ej. MedioPago.procesar() → débito, crédito o QR resuelven cada uno a su manera.

Clases, objetos e instancias

La confusión clásica del primer año

CLASE

Plantilla / molde. Define qué atributos y operaciones tienen todos los objetos de ese tipo.

OBJETO

Una cosa concreta del dominio que existe y tiene identidad propia, estado y comportamiento.

INSTANCIA

Sinónimo práctico de objeto. Un objeto es una instancia particular de una clase.

EJEMPLO · ChangoMás

Clase Clienteatributos: `dni, nombre, medioPago`operaciones: `escanear(), pagar(), pedirAyuda()`

Se instancia en tres objetos concretos:

```
cliente1 = Cliente("Pérez", "38123456", Débito)
cliente2 = Cliente("Gómez", "40987654", QR)
cliente3 = Cliente("Ruiz",  "35444555", Crédito)
```

Tres instancias distintas de la misma clase.

Responsabilidades y colaboraciones

Pensar OO = asignar quién sabe qué y quién pide qué a quién

RESPONSABILIDAD

Qué sabe y qué hace un objeto

- **Saber:** datos que mantiene (estado)
- **Hacer:** operaciones que puede ejecutar

Una clase = pocas responsabilidades, muy claras (SRP). Si una clase "hace de todo" → señal de mal diseño.

Ej. Transacción ChangoMássabe: items, total, medioPago, estado
hace: agregarItem(), quitarItem(), calcularTotal(), pagar()

COLABORACIÓN

Cómo los objetos se piden cosas entre sí

- Un objeto **NO** lo resuelve todo solo
- Delega en otros objetos que sí tienen la info/permiso
- Relaciones: asociación, agregación, composición, herencia
- Menos acoplamiento → sistema más mantenible

Ej. Transacción le pide a MedioPago que procese el cobro. No sabe si es débito o QR. MedioPago, a su vez, colabora con el banco externo.

Introducción a UML

Unified Modeling Language · estándar de la OMG

> UML es un lenguaje visual de propósito general para visualizar, especificar, construir y documentar sistemas orientados a objetos.

LO QUE ES

- Un lenguaje: notación + reglas
- Un estándar (OMG, 1997+)
- Independiente del lenguaje de programación
- Aglutina enfoques previos (Booch, Rumbaugh, Jacobson)

LO QUE NO ES

- NO es un lenguaje de programación
- NO es una herramienta específica
- NO es una metodología ni proceso
- NO garantiza un buen diseño por sí solo

PARA QUÉ LO USAMOS

→ Comunicar diseño a no-devs

→ Razonar antes de codear

→ Documentar decisiones

→ Dejar rastro que sobreviva al proyecto

Los diagramas UML · vista general

14 diagramas · este curso cubre los marcados en naranja

DIAGRAMAS ESTRUCTURALES

qué compone al sistema (estático)

- Clases · **cubierto**
- Objetos
- Componentes
- Despliegue
- Paquetes
- Estructura compuesta
- Perfiles

DIAGRAMAS DE COMPORTAMIENTO

cómo se comporta el sistema (dinámico)

- Contexto (caja negra nivel 0) · **cubierto**
- Casos de Uso · **cubierto**
- Secuencia del Sistema (SSD) · **cubierto**
- Estados · **cubierto**
- Secuencia (detallado)
- Actividades
- Comunicación / Tiempos

Estructural vs Comportamiento

Dos preguntas distintas · dos familias de diagramas

ESTRUCTURAL

¿Qué hay? ¿Cómo está compuesto?

- Foto fija del sistema
- Clases, atributos, relaciones
- Qué objetos existen y cómo se enlazan
- No importa el orden ni el tiempo

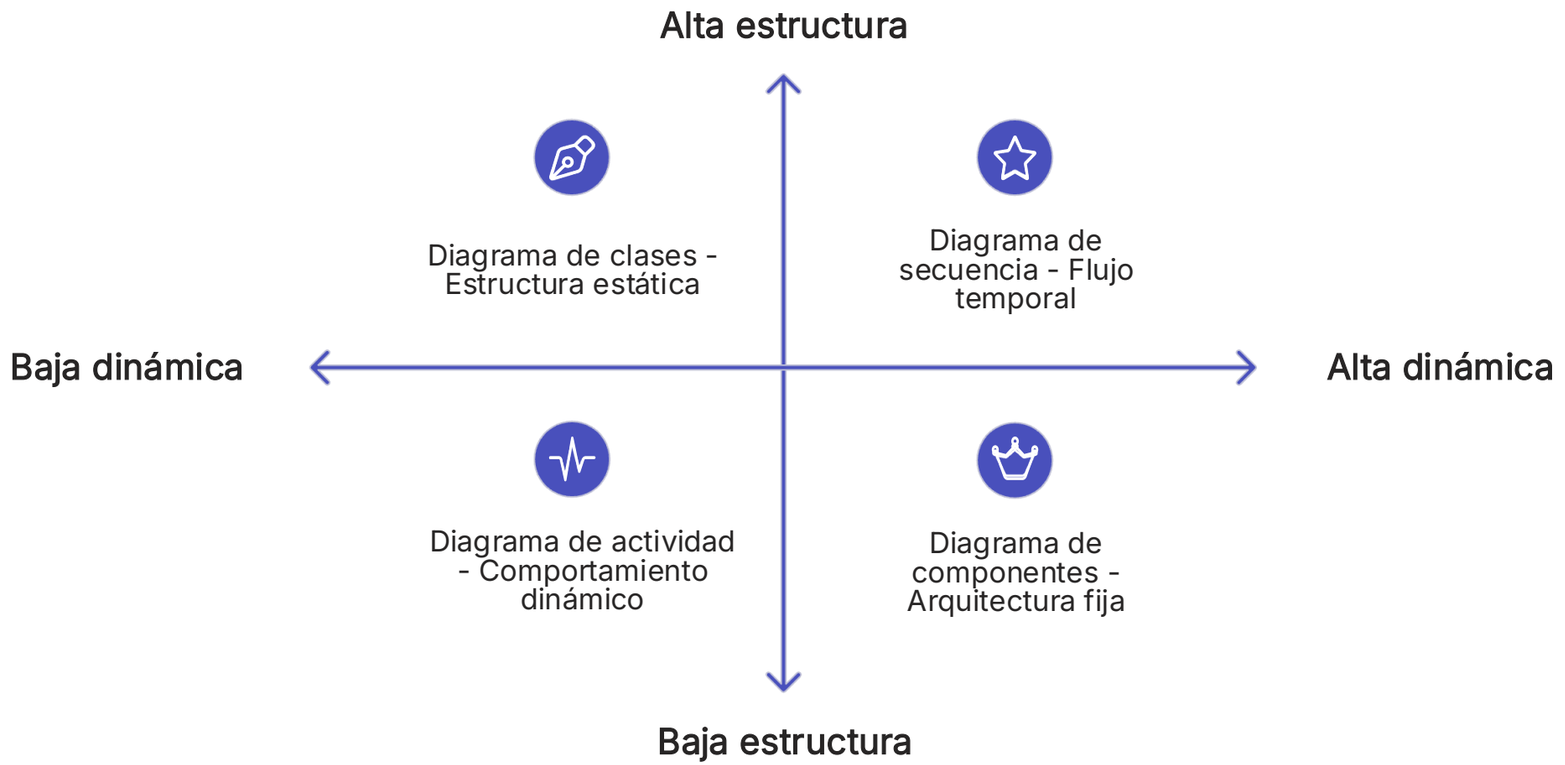
Típicos: Clases · Objetos · Componentes · Despliegue

COMPORTAMIENTO

¿Qué hace? ¿Cómo interactúa?

- Película: qué pasa en el tiempo
- Mensajes, estados, transiciones
- Quién le pide qué a quién y cuándo
- Captura flujos y variantes

Típicos: Contexto · Casos de Uso · SSD · Estados · Secuencia · Actividades



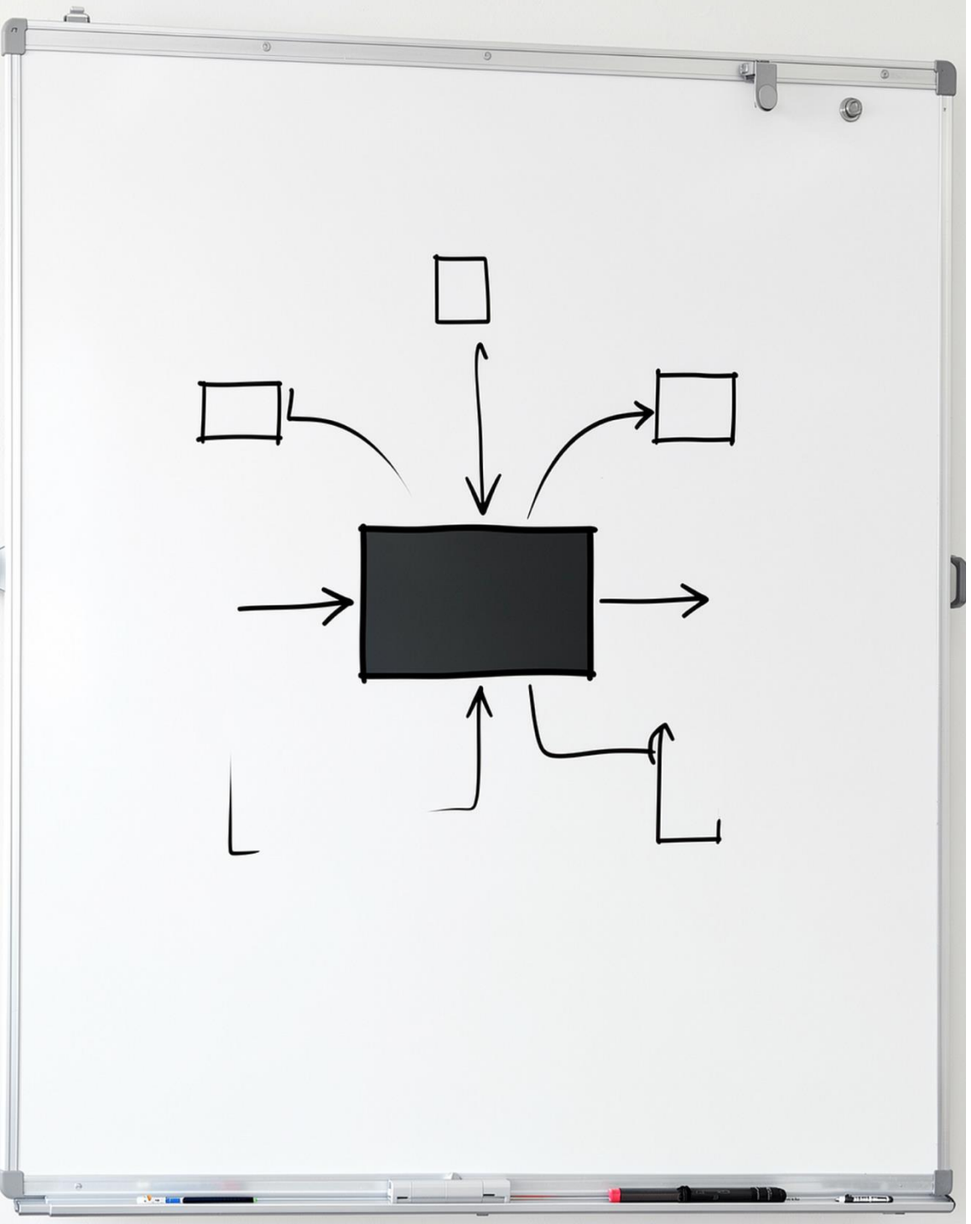


Diagrama de Contexto

¿Qué es y para qué sirve?

> El Diagrama de Contexto dibuja al sistema como una **caja negra** rodeada de todos los actores y sistemas externos con los que intercambia información.

Fijar el ALCANCE

Qué queda adentro del sistema y qué queda afuera. Lo primero que hay que decidir.

Identificar INTERFACES

Todos los puntos por donde entra o sale información. Ningún actor debe quedar oculto.

Validar con el cliente

Un diagrama simple que hasta el sponsor no-técnico puede leer y corregir.

Input para Casos de Uso

Cada actor del contexto se transforma en actor de uno o más casos de uso (clase 12).

Elementos del Diagrama de Contexto

Solo 4 tipos de cosas · simple y contundente

1

Sistema en estudio

Una sola caja en el centro. Representa TODO el software que estamos construyendo.

Notación: Un único rectángulo con el nombre del sistema.

2

Actores humanos

Personas con un rol que usan el sistema. Pueden ser varios actores distintos.

Notación: Figura stickman o rectángulo, con el rol como etiqueta.

3

Sistemas externos

Otros sistemas con los que se integra (medios de pago, inventario, cámaras).

Notación: Rectángulo con doble borde o etiqueta "<<sistema>>".

4

Flujos de información

Qué datos viajan en cada dirección. Etiquetar cada flecha con el contenido.

Notación: Flechas etiquetadas. Una flecha = una dirección de datos.

Actor primario vs secundario

Humano vs sistema · ¿quién inicia? ¿quién colabora?

ACTOR PRIMARIO

- Inicia el flujo · obtiene el objetivo del sistema
- Es quien "empuja" el caso de uso
- El sistema existe para servirlo

Ej. ChangoMás: Cliente, Asistente

ACTOR SECUNDARIO

- Colabora · el sistema le pide ayuda
- Es consultado o notificado por el sistema
- Mantiene o administra el sistema

Ej. ChangoMás: Sist. Medios de Pago, Cámaras IP

ACTOR HUMANO

- Persona con rol definido (no un individuo)
- Un mismo humano puede ser varios actores (ej. Gerente también es Cliente)
- Nombrar el ROL, no el nombre propio

ACTOR SISTEMA / DISPOSITIVO

- Software externo con el que me integro (API, servicio)
- Hardware con entradas/salidas digitales (báscula, cámara, impresora)
- Generalmente son actores secundarios

Cómo identificar actores

Checklist · 8 preguntas para no dejar ninguno afuera

01

¿Quién inicia eventos con el sistema?

Cliente, operador, administrador...

02

¿Quién provee o usa información?

Usuarios finales que leen/escriben datos.

03

¿A quién le interesa algún resultado?

Gerentes, auditores, reguladores.

04

¿Quién administra o mantiene el sistema?

Admins, operadores de sistemas.

05

¿Qué otros sistemas se integran?

Pagos, ERP, correo, autenticación.

06

¿Qué hardware/sensores envían datos?

Básculas, cámaras, lectores, IoT.

07

¿Hay procesos automáticos (schedulers)?

Cron jobs, triggers, batch.

08

¿Qué roles del organigrama tocan el sistema?

Mapa organizacional como checklist.

Antipatrones del Diagrama de Contexto

Errores que se repiten · qué NO hacer

✕ Modelar personas concretas en vez de roles

Usar "Juan Pérez" en vez de "Cliente". El diagrama tiene que sobrevivir a que Juan renuncie.

✕ Confundir módulos internos con actores

Poner "Módulo de Facturación" como actor. Si está adentro del sistema, no es actor.

✕ Flechas sin etiquetar

Una flecha "Cliente → Sistema" sin decir qué viaja (producto escaneado, pago, consulta...) no vale.

✕ Usar el diagrama como mini arquitectura

Dibujar la DB, el frontend, el balanceador. Eso es diagrama de despliegue, no de contexto.

✕ Omitir actores secundarios

Solo poner al Cliente y olvidar la API de pagos, el inventario, las cámaras. Todos esos son actores.

✕ Demasiados flujos en una sola flecha

"Datos" como etiqueta. Si son varios flujos distintos, usá varias flechas etiquetadas.

Notación simple

Caja negra nivel 0 · al alcance de un pizarrón

CONVENCIONES

Elemento	Notación
Sistema en estudio	Una caja central con el nombre del sistema
Actor humano	Stickman o rectángulo con el rol como etiqueta
Sistema/dispositivo externo	Rectángulo con doble borde o <<sistema>>
Flujo de información	Flecha etiquetada con el contenido que viaja

Una flecha = una dirección de datos. Dos direcciones → dos flechas.

REGLA DE ORO

⚠ Si el diagrama no entra en una lámina, está mal. Mantenerlo simple es parte del diseño.

Una caja central

El sistema en estudio

Pocos actores

Solo los que interactúan directamente

Flujos claros

Etiquetados con verbo + sustantivo



Ejemplo ChangoMás

Paso 1 · ¿Cuál es el sistema en estudio?

CONTEXTO DEL CASO

Grupo GDN adquirió los locales de Walmart Argentina y los relanzó como ChangoMás. La sucursal Ruta 20 tiene un sector de cajas de autoservicio que usa ~40% de los clientes.

PROBLEMAS PRINCIPALES

- Velocidad baja (31s/producto)
- Poco control del escaneo (altura de mostradores)
- Exceso de personal requerido
- Robos e inseguridad recurrentes
- Productos restringidos sin protocolo

SISTEMA EN ESTUDIO

Sistema de Caja de Autoservicio ChangoMás

Alcance: desde el escaneo y pago del cliente hasta el control posterior de productos.

ChangoMás · Identificación de actores

9 actores · 4 humanos primarios/secundarios + 5 sistemas/hardware

ACTORES HUMANOS (4)

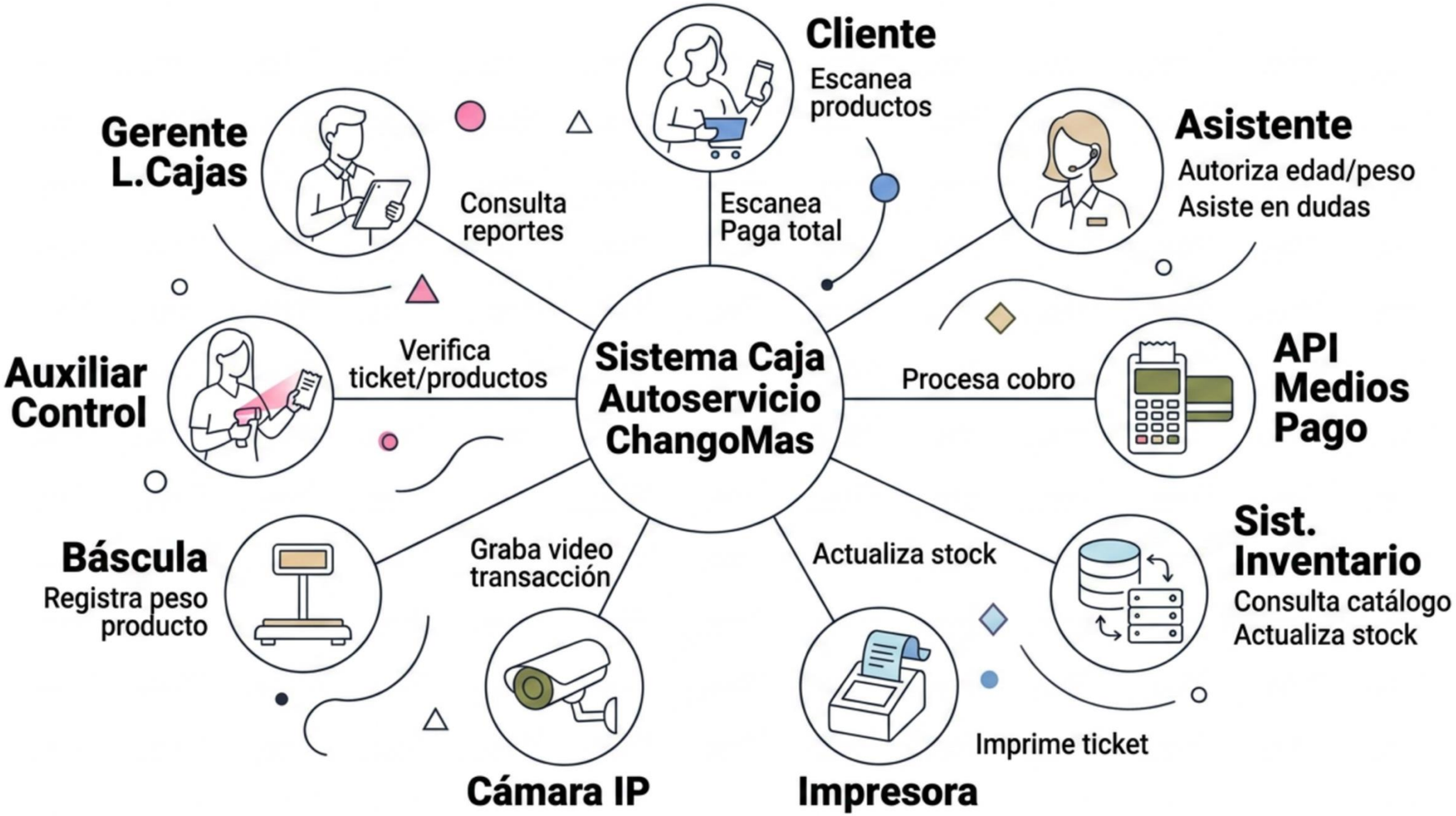
#	Actor	Tipo	Rol
1	Cliente	Primario	Escanea, paga, retira mercadería
2	Asistente de Autoservicio	Primario	Coordina, asiste, autoriza restringidos
3	Auxiliar de Control	Primario	Verifica ticket vs. mercadería post-pago
4	Gerente de Línea de Cajas	Secundario	Consulta reportes, supervisión

SISTEMAS Y HARDWARE (5)

#	Actor	Tipo
1	Sist. Medios de Pago	Externo · débito, crédito, QR, transferencia
2	Sist. de Inventario / POS	Externo · catálogo y registro de ventas
3	Báscula	Hardware · verifica peso de productos
4	Cámara IP	Hardware · video de cada transacción
5	Impresora de Ticket	Hardware · emite comprobante físico

ChangoMás · Diagrama de Contexto

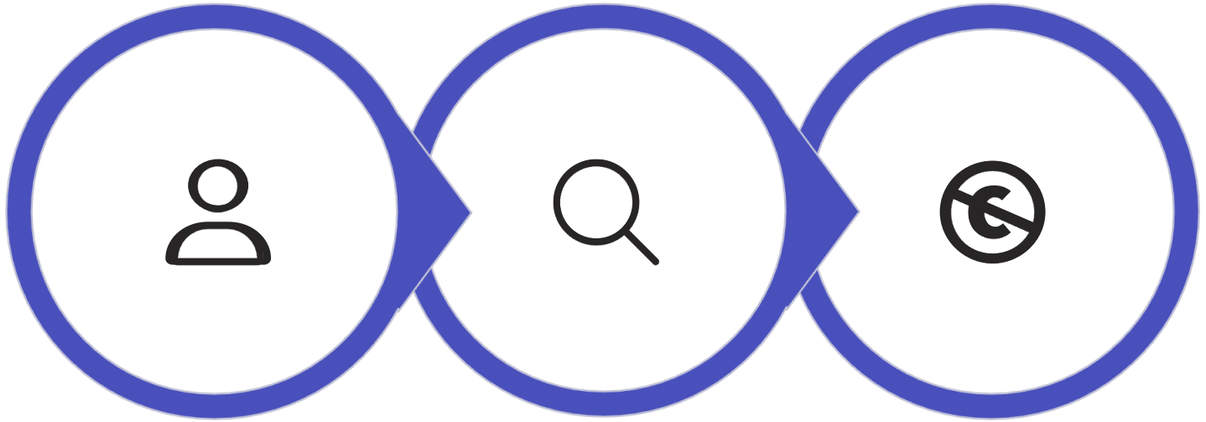
Nivel 0 · sistema en estudio + 9 actores con flujos etiquetados



Flechas = flujos de información · etiquetadas con el contenido. El sistema es la caja negra central; todos los actores quedan fuera del alcance.

Contexto → Casos de Uso → Dominio

Secuencia de modelado en el bloque UML Análisis



Clase 11 (hoy) · CONTEXTO

Actores + flujos + alcance. La caja negra.
Pregunta clave: Quién interactúa con el sistema y con qué.

Clase 12 · CASOS DE USO

Objetivos de cada actor + flujos principales y alternos.
Pregunta clave: Qué hace el sistema para cada actor.

Clase 13 · DOMINIO

Entidades del negocio, atributos, relaciones.
Pregunta clave: Qué conceptos/datos maneja el sistema.

De la HU al actor del Contexto

Scrum y UML se complementan, no compiten

HISTORIA DE USUARIO · HU-011

> Como **Cliente**, quiero **escanear productos** con el lector de código de barras, para cargarlos rápido al ticket sin ayuda del cajero.

CRITERIOS DE ACEPTACIÓN (resumen)

- Código válido → se agrega al ticket
- Código inválido → mensaje en pantalla
- Respuesta <1 seg (RNF-01)

MAPEO AL DIAGRAMA DE CONTEXTO

La HU aporta:
rol ("Cliente") → actor primario
objetivo ("escanear") → flujo etiquetado
criterio técnico (catálogo) → actor sistema externo

→ **Cliente** → actor primario en el diagrama

→ Flujo **"escanea"** → flecha etiquetada hacia el sistema

→ **Catálogo de Productos** → actor sistema externo (Inventario/POS)

Errores típicos al dibujar Contexto

Top-5 que veo año tras año en entregas de alumnos

1 Inflar el sistema

La caja negra central se agranda hasta incluir actores que son EXTERNOS al alcance definido.

2 Mezclar niveles de abstracción

Un actor es "Cliente" y el otro es "Módulo de autenticación". Hay que quedarse en un solo nivel.

3 No etiquetar flechas o etiquetar genérico

"Datos", "Info", "Comunica". Eso no dice nada. Etiquetar con el verbo/sustantivo específico.

4 Olvidar actores sistema

Solo ponen humanos y olvidan APIs, hardware, schedulers. Usar el checklist de 8 preguntas.

5 Diagrama como árbol de módulos

Confundir Contexto con diagrama de componentes. Si dibujás lo de ADENTRO, ya no es contexto.

Herramientas para dibujar

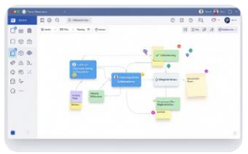
Elegir una · mantenerla durante todo el cuatrimestre



Draw.io (diagrams.net)

⭐ Recomendado

- Gratis, online o local
- Importa/exporta XML editable
- Librería UML completa
- Se integra con Git (XML versionable)



Lucidchart

ALTERNATIVA WEB

- Colaboración en tiempo real
- Plantillas prearmadas
- Integración con Confluence/Teams
- Gratis hasta 3 diagramas



Visual Paradigm

PROFESIONAL

- Soporte completo UML 2.5
- Generación de código / ingeniería inversa
- Requiere licencia (Community gratis limitada)



Mermaid.js

DEVS / MARKDOWN

- Diagramas como código (.mmd)
- Se renderiza en GitHub/VSCode
- Ideal para docs técnicas
- Mejor para flujos/CU que para Contexto

Ejercicio en clase — Diagrama de Contexto de su OAS

20 minutos · trabajo grupal por caso propio

1

Identificar el sistema en estudio

Releer el mandato ampliado. ¿Cuál es el alcance EXACTO?

2

Listar actores con el checklist

Aplicar las 8 preguntas. Separar primarios/secundarios, humanos/sistema.

3

Dibujar el diagrama

Caja central + actores + flechas etiquetadas. En papel o Draw.io.

4

Validar contra antipatrones

Revisar la lámina de antipatrones antes de cerrar.

ENTREGABLE AL FINAL DE LA CLASE: Foto del diagrama + lista de actores primarios y secundarios (puede ser en cuaderno, Draw.io o pizarrón).



Tarea para casa

Refinar el Diagrama de Contexto antes de Clase 12

A · Pasar el diagrama a limpio en Draw.io

PNG + fuente editable en el repo del grupo.

B · Completar la tabla de actores

Nombre · Tipo (humano/sistema) · Primario/Secundario · Responsabilidad.

C · Etiquetar cada flecha con su flujo específico

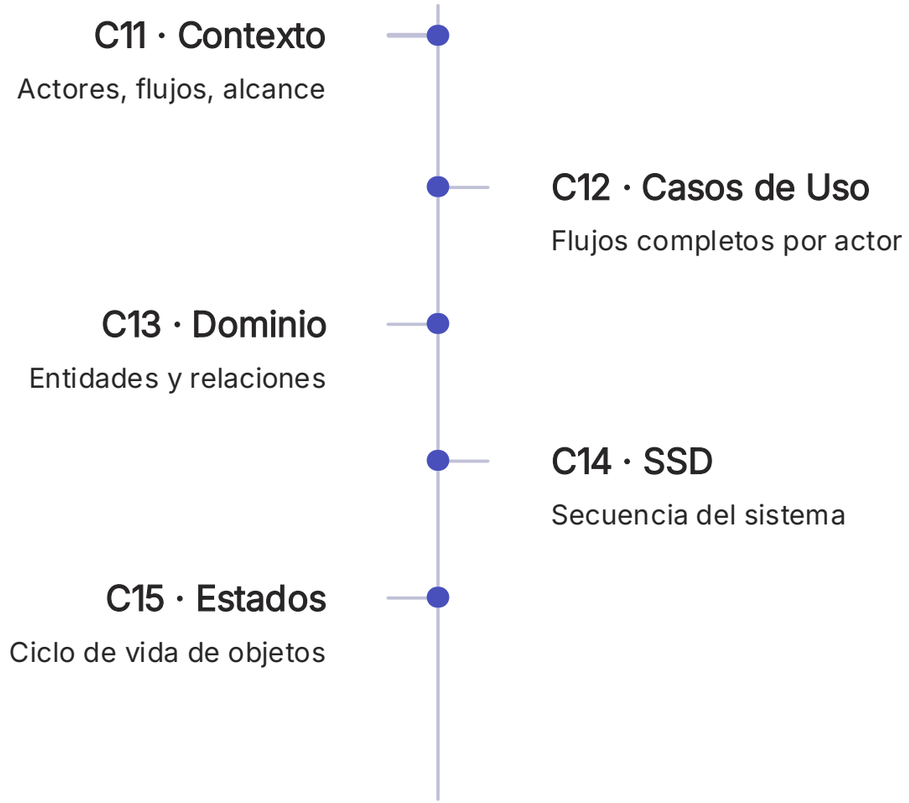
Evitar genéricos como "datos" o "info". Verbo + sustantivo.

D · Marcar las HU del sprint 1 asociadas a cada actor

Servirá de puente para la clase 12 cuando armemos Casos de Uso.

Próxima clase — Casos de Uso

Clase 12 · del actor al flujo completo



LO QUE VEREMOS

- Diagrama de Casos de Uso UML
- Relaciones: include, extend, generalización
- Plantilla de Especificación de CU (Cockburn)
- Ejemplo: UC-01 "Escanear y pagar" en ChangoMás

LO QUE TRAEN USTEDES

- Diagrama de Contexto final del OAS propio
- Tabla de actores con tipo y rol
- Mapeo actor ↔ HU del sprint 1
- Listo para arrancar sus propios CU en clase

Recursos

Para profundizar fuera del aula

📘 Booch, Rumbaugh, Jacobson — The Unified Modeling Language User Guide

El clásico de UML. Capítulos 1-3 cubren OO y UML intro.

📗 Craig Larman — Applying UML and Patterns (3ra ed.)

El texto de referencia para casos de uso, dominio y GRASP. Muy didáctico.

📙 Alistair Cockburn — Writing Effective Use Cases

Plantillas y ejemplos de CU a trazo fino. Corto y práctico.

📄 UML 2.5 Specification — omg.org/spec/UML

Documento oficial. Largo y denso — consultar solo ante duda específica.

🛠 Draw.io — diagrams.net

Gratis, soporta toda la notación UML que usamos en el curso.

🛠 Visual Paradigm Community Edition

Licencia gratis para estudiantes. Soporte completo UML 2.5.

¿Preguntas?

CLASE 11

💡 Idea 1

OO es pensar en **responsabilidades y colaboraciones**, no en clases sueltas.

💡 Idea 2

El Diagrama de Contexto fija el **ALCANCE** y los **ACTORES** antes de cualquier otra cosa.

💡 Idea 3

Cada **flecha etiquetada** es una pregunta menos que tendremos que hacer al cliente.

Ing. Fernando Bono · linkedin.com/in/fernandobono/ · MyD 2026

